

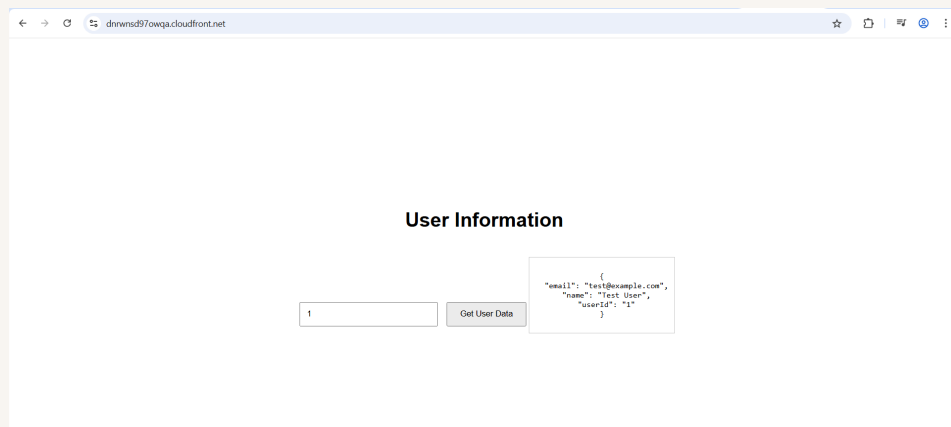


nextwork.org

Build a Three-Tier Web App



anshgupta5667@gmail.com





anshgupta5667@gmail...

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will demonstrate how to set up a three-tier web app from scratch. First start with the presentation tier, then logic tier, then data tier and then tying them all together.

Tools and concepts

Services I used were Amazon S3, Cloudfront, DynamoDB, Lambda , API Gateway. Key concepts I learnt include Lambda functions, CORS errors, updating the javascript file with the API invoke URL, and testing the invoke url with in the browser.

Project reflection

This project took me approximately 4 hours. The most challenging part was to resolving the CORS errors in both API Gateway and Lambda.

I chose to do this project today because to learn about Three-tier architecture and set up our own web app.



anshgupta5667@gmail...

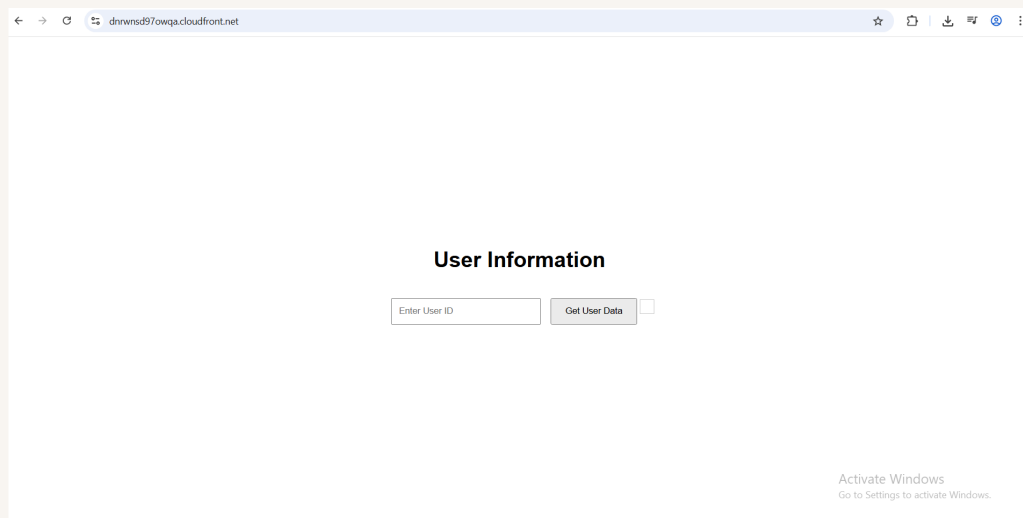
NextWork Student

nextwork.org

Presentation tier

For the presentation tier, I will set up how website will be visible to our end users. Because presentation tier is responsible for our website files (s3 bucket) + website distribution.(cloudfront)

I accessed my delivered website using cloudfront distribution url. We set up an Origin Access Control that that lets our S3 bucket to restrict access to only Cloudfront distribution





Logic tier

For the logic tier, I will set up Lambda function to handle request (find user by id) and also an API with API Gateway to receive request from user and pass it to Lambda.

The Lambda function retrieves data by userId , the user enters on the web app , in DynamoDB. The AWS SDK is used in the lambda function to so that we can use templates and libraries that let us find the correct DynamoDB table and request data.

The screenshot shows the AWS Lambda console for the 'RetrieverUserData' function. The code is written in JavaScript and uses the AWS SDK to interact with DynamoDB. The function handler takes an event object and extracts the 'userId' from the query string parameters. It then uses the 'dynamodb' package to create a 'DynamoDBClient' and a 'DynamoDBDocumentClient'. The handler uses the 'ddbClient' to get the item from the 'UserData' table. If the item is found, it returns a 200 status code with the item's data in the body. If not found, it returns a 404 status code. The console also shows the 'Deploy' button and a 'Test' button. A notification at the bottom indicates 'Successfully updated the function RetrieverUserData.'

```
1 // Import individual components from the dynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4
5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         body: JSON.stringify(Item),
22         headers: { 'Content-Type': 'application/json' }
23       };
24     } else {
25       return {
26         statusCode: 404,
27         body: 'User not found'
28       };
29     }
30   } catch (error) {
31     console.error('Error retrieving user data:', error);
32     return {
33       statusCode: 500,
34       body: 'Internal server error'
35     };
36   }
37 }
```




Data tier

For the data tier, I will set up DynamoDB database that store user data because we are using api to find user data through lambda.

The partition key for my DynamoDB table is userId which means when our table looks for data , it will look based on userID. It can return all data(values) related to that item with that ID.

The screenshot shows the AWS DynamoDB console interface. On the left, there's a navigation menu with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main area displays the 'Explore Items' view for the 'UserData' table. It includes a search bar, a table selection dropdown, and a 'Select attribute projection' dropdown set to 'All attributes'. Below this, there's a 'Filters - optional' section with 'Run' and 'Reset' buttons. A green status bar indicates 'Completed - Items returned: 0 - Items scanned: 0 - Efficiency: 100% - RCUs consumed: 2'. The table 'Table: UserData - Items returned (3)' shows a scan started on January 07, 2026, at 15:28:59. The table has three columns: 'userid (String)', 'email', and 'name'. The items are listed as follows:

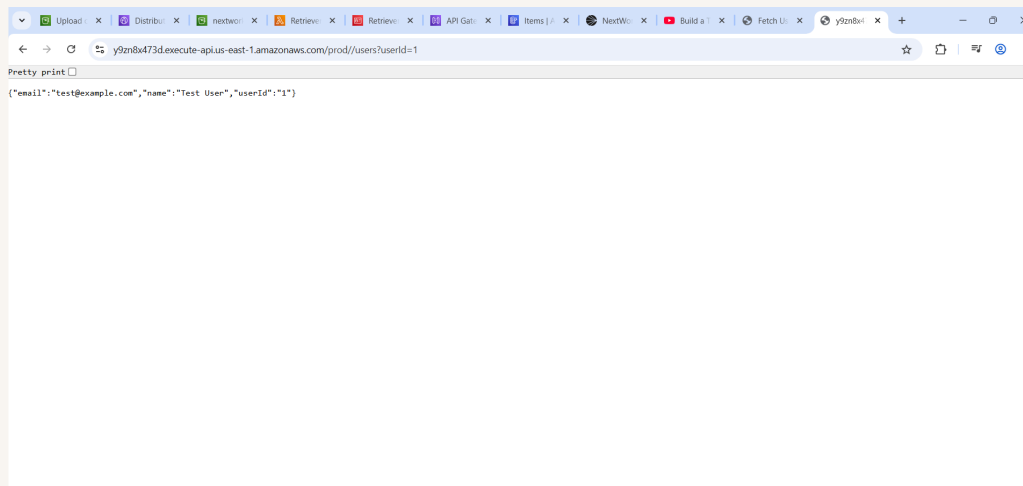
userid (String)	email	name
3	test3@exa...	Test User 3
2	test2@exa...	Test User 2
1	test@exam...	Test User



Logic and Data tier

Once all three layers of my three-tier architecture are set up, the next step is to connect the presentation and logic tier. This is because currently there is no way for API to catch requests that users make through our distributed site.

To test my API, I visited the invoke URL of the prod stage API. This let us test whether we can use the API and retrieve the user data. The results were some user data in JSON when we looked up userid = 1. This proved logic + data tier connection.





anshgupta5667@gmail...

NextWork Student

nextwork.org

Console Errors

The error in my distributed site was because there was an error with script.js file (file that i uploaded into S3). The script.js file is referencing an prod stage API url placeholder and my API's actual URL.

To resolve the error, I updated script.js by replacing some placeholder text with the API's prod stage INVOKE URL. I then reuploaded it into S3 because the s3 bucket still storing the old script.js file.

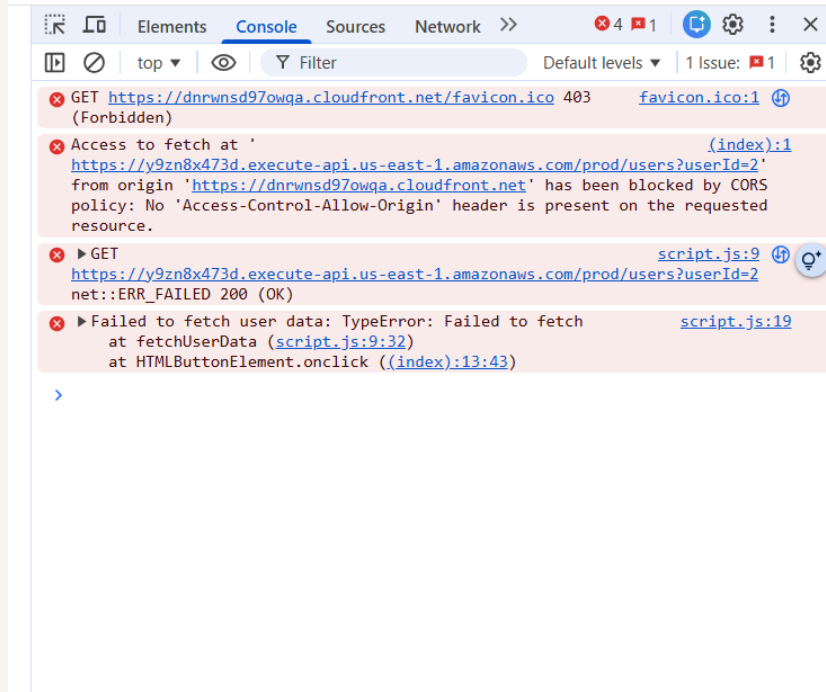
I ran into a second error after updating script.js. This was an error with CORS because we have not allowed API Gateway to allow request from CloudFront Distribution



anshgupta5667@gmail...

NextWork Student

nextwork.org





Resolving CORS Errors

To resolve the CORS error, I first went into our API and enabled CORS on the /users resource. We then made sure GET requests are enabled and referenced our Cloudfront domain as domain getting access.

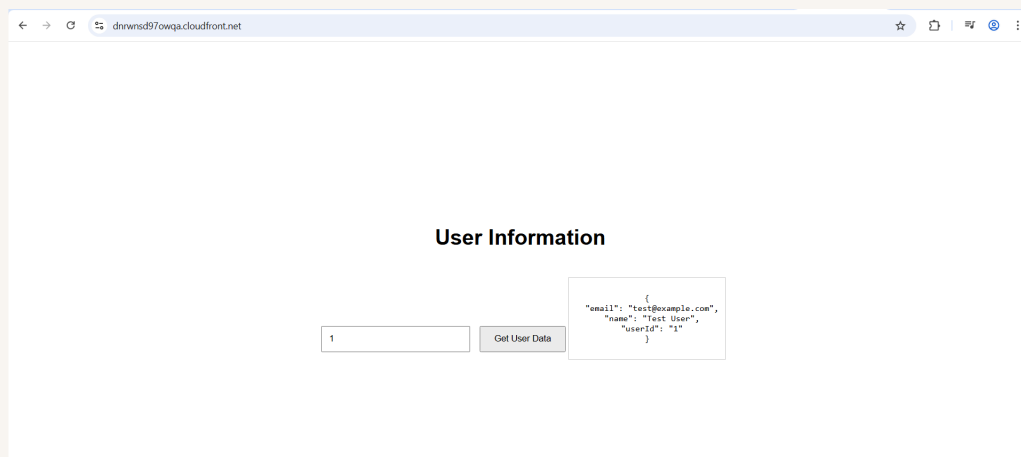
I also updated my Lambda function because it needs to be able to return CORS headers to show that it has the permissions to invoke the API's URL and return a response. We added 'Access-Control-Allow-Origin' as the header.

```
8 async function handler(event) {
16   const command = new GetCommand(params);
17   const { item } = await db.send(command);
18   if (item) {
19     return {
20       statusCode: 200,
21       body: JSON.stringify(item),
22       headers: { 'Content-Type': 'application/json',
23                 'Access-Control-Allow-Origin': 'https://dnrwnsd97owqa.cloudfront.net'
24     };
25   }
26   } else {
27     return {
28       statusCode: 404,
29       body: JSON.stringify({ message: "No user data found" }),
30       headers: { 'Content-Type': 'application/json',
31                 'Access-Control-Allow-Origin': 'https://dnrwnsd97owqa.cloudfront.net'
32     };
33   }
34   }
35   } catch (err) {
36     console.error("Unable to retrieve data:", err);
37     return {
38       statusCode: 500,
```



Fixed Solution

I verified the fixed connection between API Gateway and CloudFront by looking the user data in the distributed site again. In my final test , user data could be returned- so a user request in the presentation tier gets data from the data tier.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

